How You Build the Queue Decides the Winner: Re-Evaluating Duplicate Pull Request Detection on Coding-Agent PRs

Harshul Jain, Saurabh Yergattikar, Tanya Sah
Independent Researchers

***Abstract*—**Coding agents open many pull requests (PRs) that repeat work already waiting for review. A duplicate-PR detector must spot such a PR when it opens, by comparing it with the PRs that are open in the repository at that moment. We test how well detectors do this on agent PRs, using 1767 duplicate pairs (313 opened by different authors) and 2965 PRs with no duplicate from the public AIDev dataset. We compare 67 strategies, from word matching and existing bots to neural text models and large language models (LLMs). Our main finding is that the way the test queue is built decides the winner. If the queue also keeps PRs closed in the last seven days, a learned detector ranks the right PR first +9.1 percentage points more often than a classic research detector. If the queue holds only open PRs, as a real tool sees it, the learned detector, the classic detector and a simple rule that checks for a shared issue or a link all rank the right PR first 0.911–0.936 of the time, with no clear winner. The methods differ more in false alarms (1.2% for the link rule and 1.8% for the learned detector, against 5.5% for the best existing bot) than in ranking. Duplicates are also costly: 15% of cross-author duplicates are noticed only after a day. Duplicates that change different files remain hard for every method.

I. Introduction

AI coding agents such as Codex, Copilot, Devin, Cursor and Claude Code now open hundreds of thousands of pull requests (PRs) on GitHub [1]. Many are never merged, and duplicated work is a leading reason. In a hand-labelled sample of 600 failed agent PRs, 142 (23.7%) were duplicates of other PRs [2], and "resolved by another PR" is the most common reason why agent fix-PRs stay unmerged [3]; see also [4], [5].

Detecting duplicate PRs is an established research problem [6], [7], [8], [9], [10], [11], and several small bots now offer it on GitHub. Most detectors were scored on balanced sets of random PR pairs [7]. A deployed tool faces a different task: when a PR opens, it must rank the PRs that are open at that moment and decide whether to warn anyone. Work on duplicate bug reports showed that unrealistic set-ups can make methods look better than they are [12]. We find that the same is true for PRs, and that even a "realistic" set-up can mislead if the queue is not built exactly as the tool would see it.

A queue that also keeps PRs closed in the last seven days looks harmless, and our own first benchmark used it. It favours any method that knows whether the older PR is still open, and under it a learned detector, Jinx [13], wins clearly. With open-only queues the advantage disappears. We report both designs because the contrast is itself a finding about how duplicate detectors should be evaluated.

Our contributions are:

- a measurement of what duplicate agent PRs cost maintainers: how long they go unnoticed and how much review they absorb;
- *LiveQueue*, a benchmark of 1767 duplicate pairs and 2965 PRs without a duplicate, built from 76,209 agent PRs, where each new PR comes with the PRs open at that time;
- a comparison of 67 strategies, including three open-source duplicate-PR bots and five LLM judges, with repository-level confidence intervals; and
- evidence that two evaluation choices, random pairs and recently closed PRs in the queue, change which method appears best.

II. Research Questions

RQ1 What do duplicate agent PRs cost, and how long do they go unnoticed?
RQ2 How well do detectors find duplicates in the open-only queue, and how often do they raise false alarms?
RQ3 How much do evaluation choices change the conclusions?
RQ4 Which duplicates remain hard, and do LLMs help?

III. The LiveQueue Benchmark

A. Data source

We use AIDev v5 [1], a public dataset (CC BY 4.0) of PRs opened by coding agents on GitHub up to 31 March 2026. For each PR it provides the title and description, comments, reviews, linked issues, the changed files and the code changes.

B. Finding duplicate pairs

Maintainers and authors often say why they close a PR, for example "superseded by #123". We search PR comments for five phrases (*duplicate of*, *superseded by*, *in favor of*, *replaced by*, *same as*) followed by a PR number, as earlier datasets did [14], [11]. No person checked each pair, so we call these *silver* labels. In each pair, the newer PR is the *query* and the older PR is the duplicate the tool should find.

C. Building the queue

We use two names for queue designs throughout. The *open-only queue* of a query holds the PRs in the same repository that were open when the query was created, capped at the 200 most recent (what a GitHub Action can fetch cheaply). The *7-day queue* also holds PRs that closed less than seven

days before the query. Unless stated otherwise, results use the open-only queue. We keep a pair only if its duplicate is in the queue, which leaves 1767 pairs. The 200-PR cap removes 29 further pairs whose duplicate was still open (11 cross-author); a deployed tool would miss these, so we also report Recall@1 with them counted as misses. To measure false alarms we add 2965 *no-duplicate queries*: merged PRs from the same repositories with no duplicate-like comment. A good tool should stay quiet on these. Link signals use only the query's own title and description, because the older PR's text in the dataset may have been edited after the duplicate was found.

In RQ3 we compare with the 7-day queue ($n = 1867$ pairs, 342 cross-author).

D. Relations and kinds of duplicates

Of the 1767 pairs, 1454 are *self-replacements*, where the same author opened both PRs, and 313 are *cross-author* pairs (207 with the same agent, 106 with different agents). We focus on cross-author pairs, because a maintainer cannot spot them from the author's name. We also sort pairs by shared code: (A) *code twin*, at least half of the changed lines identical; (B) *partial overlap*; (C) *same files, different code*; and (D) *no shared files*, where the PRs reach the same goal through different files.

E. How good are the labels?

Some silver labels are not true duplicates; a small fix may be closed "in favor of" a large PR that includes it. We asked Claude Haiku 4.5 to read each pair and label it as duplicate, replacement, subsumed, related or unclear. In 200 random pairs, 79.0% (95% CI 73.5–84.5%) were duplicates or replacements, and 97.5% when subsumed pairs are included. For the 313 cross-author pairs in our open-only sample the numbers are 84.3% (CI 80.2–88.2%) and 96.5%. This is an LLM estimate, not a human check; a human check of 100 pairs is planned.

IV. Strategies

We tested 67 strategies in seven families (Table I). INTRUDE [7] is a classic research detector that combines text, code and issue signals; we add the time between the PRs [8] and call the result *INTRUDE + time*. Jinx [13] is a logistic-regression (LR) combination of text, code and context signals that we release as a tool; here it is one learned strategy among several. The *link rule* ranks candidates by a shared linked issue plus a mention of the candidate's PR number in the new PR, and breaks ties by TF-IDF; we also report it without the tie-break. Re-rankers and LLM judges re-score the top 5 or 10 candidates from Jinx; candidates they did not score keep Jinx's score.

TABLE I
Strategy families.

Family	Examples
Simple rules	newest PR first; TF-IDF [6] and BM25 (word-overlap scores that weight rare words more) on raw text, text with agent boilerplate removed, or the diff
Code overlap	shared files, overlapping line ranges, identical added lines
Links	shared issue, mention of the other PR's number
Embeddings	small neural models that map text to vectors: MiniLM, BGE, UniXcoder, CodeBERT, Jina-code
Existing bots	contrib-guard (exact port), Simili and PRSense (re-implemented with local models)
Learned	LR or gradient boosting over INTRUDE + time, Jinx, or all signals
Re-rankers, LLM judges	three cross-encoders and a fine-tuned BGE re-ranker (read both PRs together); Qwen2.5 1.5B/7B/Coder-7B/14B and Claude Haiku 4.5 asked whether two PRs are duplicates

V. How We Measure

All learned strategies use five-fold cross-validation in which each fold holds out whole repositories. *Recall@1* is the share of queries whose true duplicate is ranked first, and *Recall@5* the share where it is in the top five.

For false alarms, each strategy gets a score threshold from its training folds (the median top score of cross-author duplicates), and we count how often the top candidate of a held-out no-duplicate query passes it. Recall@1 is a ranking measure; at this threshold a tool actually *flags* only about half of the duplicates.

All 95% confidence intervals (CIs) use 2,000 bootstrap samples drawn by repository. Comparisons between two methods use paired differences, with both methods scored on the same queries. CIs are not adjusted for multiple comparisons, so a single CI that just excludes zero is weak evidence; subgroup results are exploratory.

Before the study we fixed one success test: Jinx's cross-author Recall@5 must beat the strongest baseline that does not use Jinx's signals, with a 95% CI above zero. It fails in every setting. On open-only queues all strong methods reach Recall@5 of at least 0.987, so no method can show a clear gain.

VI. Results

A. RQ1: What do duplicates cost?

For each cross-author pair in our open-only sample ($n = 311$) we found the first comment by a person that marked the duplicate and measured the time from the second PR opening, the first moment a tool could warn, to that comment. For cross-author pairs the median was 29 minutes, but 44% took more than an hour, 15% more than a day and 5% more than a week (mean 39 hours). In 22% of cases a person commented on or reviewed the PR that was later closed in the window between the second PR opening and the duplicate comment: 74 human comments and 19 reviews in total. The closed PRs added a median of 126 lines; we do not know how much of that code was reused in the PR that was kept.

TABLE II

CROSS-AUTHOR PAIRS ON OPEN-ONLY QUEUES ($n = 313$) AND FALSE ALARMS ON 2965 PRS WITHOUT A DUPLICATE. BOLD: BEST IN COLUMN. †/‡: MORE/FEWER FALSE ALARMS THAN JINX, PAIRED 95% CI EXCLUDES ZERO. LAST COLUMN: ROW MINUS JINX IN RECALL@1 PERCENTAGE POINTS, PAIRED 95% CI (UNADJUSTED). RECALL@5 DIFFERENCES ARE ALL WITHIN NOISE.

Strategy	R@1	R@5	False alarms (%)	R@1: row − Jinx (pts)
Jinx				
Jinx (with MiniLM)	0.927	**0.990**	1.8	–
Jinx without MiniLM	0.917	**0.990**	1.9	-1.0 [-2.7, +0.9]
Jinx with an extra model on its top 5				
All signals, LR	0.927	**0.990**	1.6	0.0 [-1.7, +2.0]
Jinx + BGE re-ranker + Qwen-7B	**0.936**	0.978	1.4	+1.0 [-1.0, +2.6]
Qwen2.5-14B + Jinx	0.917	**0.990**	1.4	-1.0 [-2.0, 0.0]
Baselines				
Link rule, TF-IDF tie-break	**0.936**	0.978	**1.2**	+1.0 [-2.7, +4.3]
INTRUDE + time [7], [8]	0.914	0.987	2.0	-1.3 [-3.1, +0.7]
contrib-guard (bot)	0.914	0.981	5.5†	-1.3 [-4.7, +2.7]
TF-IDF [6]	0.911	0.981	3.5†	-1.6 [-4.9, +2.2]
BGE-small embedding	0.907	0.981	2.3	-1.9 [-4.6, +1.1]
Link rule, no tie-break	0.690	0.808	2.3	-23.6 [-30.3, -16.7]

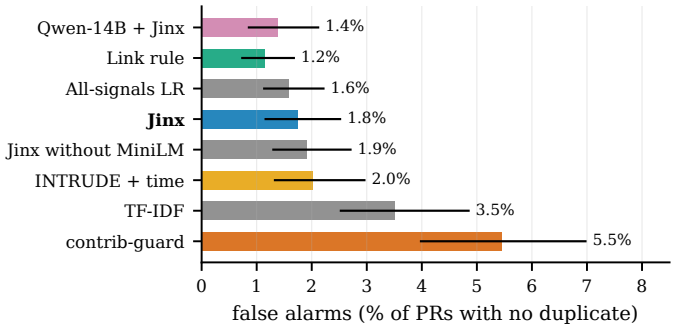


Fig. 1. False alarms on PRs with no duplicate (open-only queues), with 95% CIs. Each method uses the threshold at which it flags about half of the cross-author duplicates.

We then replayed each pair: had a detector commented when the new PR opened, at its own threshold, would it have named the right duplicate? Jinx would have done so in 46% of cases (CI 36–55%), INTRUDE + time in 47% and contrib-guard in 46%, with 1.8, 2.0 and 5.5 false alarms per 100 PRs without a duplicate. A further 2% of Jinx's warnings on these pairs named the wrong PR. The benefit is therefore not specific to one detector, and it is an upper bound: a warning still has to be read, and duplicates that no one marked are not counted.

B. RQ2: Detectors on the open-only queue

Table II gives the main results. On open-only queues, the strong methods are close: Jinx reaches 0.927, INTRUDE + time 0.914, contrib-guard 0.914, plain TF-IDF 0.911 and the link rule 0.936 (0.690 without its TF-IDF tie-break). Every Recall@1 difference between Jinx and these baselines has a CI that includes zero, and so does Recall@5 (Jinx 0.990, INTRUDE + time 0.987). Counting the 11 cross-author pairs hidden by the 200-PR cap as misses lowers Jinx's Recall@1 to 0.895.

The methods differ more in how often they raise false alarms (Fig. 1). Jinx raises 1.8% false alarms. The best existing bot raises 5.5% (difference +3.7 points, CI +2.2 to +5.1) and TF-IDF 3.5% (+1.8, CI +0.9 to +2.8). INTRUDE + time (2.0%) is not clearly different from Jinx, and the link rule (1.2%) is at least as quiet.

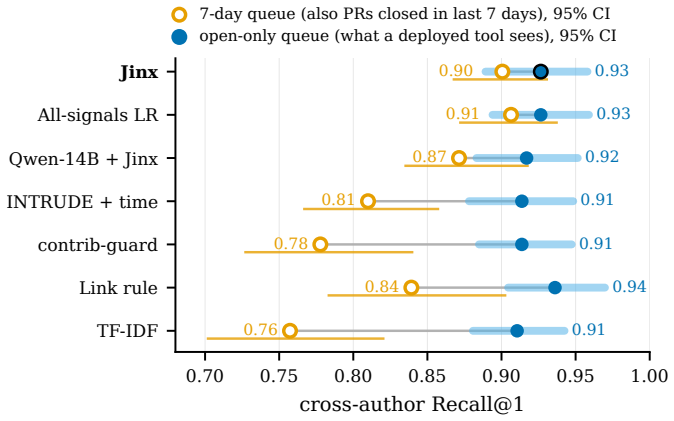


Fig. 2. Cross-author Recall@1 under the two queue designs, with 95% CIs. Orange: 7-day queue. Blue: open-only queue. The large gaps between methods in the 7-day queue vanish in the open-only queue.

C. RQ3: Evaluation choices change the winner

Fig. 2 compares the two queue designs. In the 7-day queue, Jinx reaches 0.901, INTRUDE + time 0.810, contrib-guard 0.778 and TF-IDF 0.757; Jinx's lead over INTRUDE + time is +9.1 points (CI +5.0 to +13.1). In that design closed PRs were almost always wrong answers (the true duplicate was already closed in only 18 of 342 cross-author pairs), and Jinx has a signal for "the older PR is still open". In the 7-day queue, removing that signal drops Jinx to 0.839; giving it to INTRUDE + time lifts that method to 0.892. Most of the gain came from the queue design, not from better duplicate detection.

Random pairs have the opposite problem: they make strong methods look the same (Fig. 3). The two scorings agree on the broad order (Kendall $\tau = 0.70$, a rank agreement from -1 to 1), mostly because weak methods are weak under both. Among strong methods, random-pair AUC is close to 1 and says little about which method ranks the right PR first.

D. RQ4: Hard duplicates and LLMs

Code twins are easy: every combined method ranks them first at least 0.95 of the time (Fig. 4). Duplicates that share no files (class D, $n = 25$) are harder: Jinx 0.720, INTRUDE + time 0.680. The link rule and contrib-guard (0.84) both beat Jinx here, which suggests that many intent-level duplicates are found through a shared issue or a similar description. On the 118 cross-author pairs with no link at all, Jinx reaches 0.839 and INTRUDE + time 0.805.

LLM judges, asked whether two PRs are duplicates, do not rank more duplicates first. Qwen2.5-14B averaged with Jinx on Jinx's top five reaches 0.917 (Jinx 0.927) with 1.4% false alarms (CI of the difference -0.7 to +0.0 points), so it is not clearly quieter either, and it needs a GPU. At a strict 1%

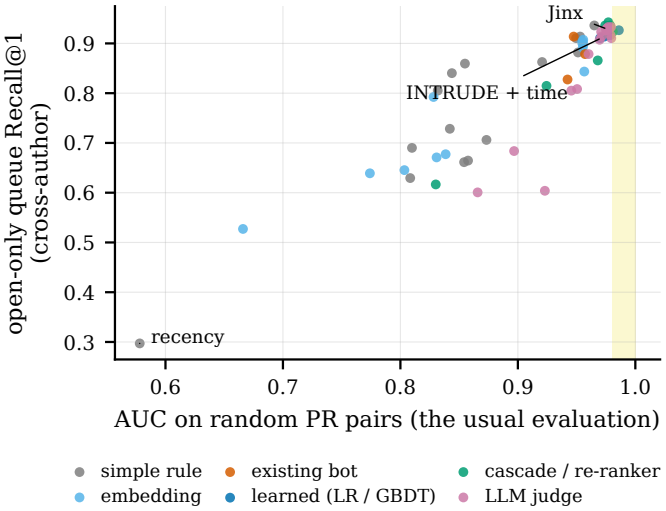


Fig. 3. Each dot is one strategy. AUC on random PR pairs (x; the chance that a random duplicate pair scores higher than a random non-duplicate pair) against open-only queue Recall@1 (y). The two agree on which methods are weak, but in the shaded band 2 combined methods have AUC above 0.98 and Recall@1 from 0.92 to 0.93.

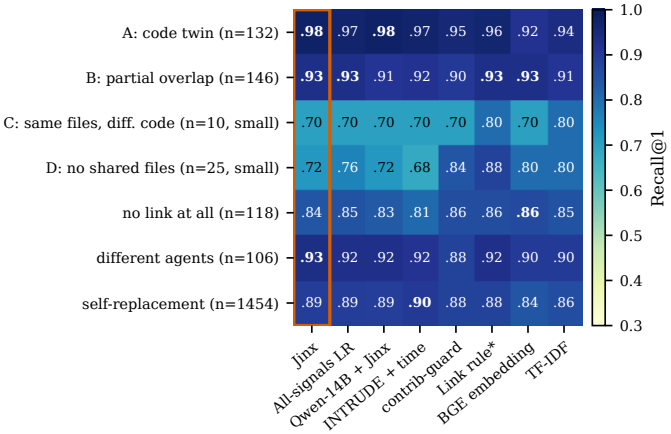


Fig. 4. Recall@1 by kind of duplicate, cross-author unless noted. Best value per row in bold (rows with $n \geq 30$ only); Jinx outlined. *The link rule falls back to TF-IDF when there is no link.

false-alarm budget the link rule catches 44% of cross-author duplicates, against 32% for Jinx and 34% for INTRUDE + time (Fig. 5). Our Claude Haiku 4.5 scores were computed for the 7-day queue and cover only 30% of the current top-5 candidates, so we do not report Haiku results for the open-only queue.

Fig. 6 shows cost. Without MiniLM, Jinx needs 6 ms per PR at the median (40 ms at the 90th percentile); with MiniLM, 0.4 s (2.3 s). Re-rankers and LLM judges add more time without a clear gain in Recall@1.

VII. Threats to Validity

Labels. Our labels come from comments, so unmarked duplicates are missed and some no-duplicate queries may hide real duplicates. The LLM audit estimates label quality but is not a human check, and on the 264 cross-author pairs it judged

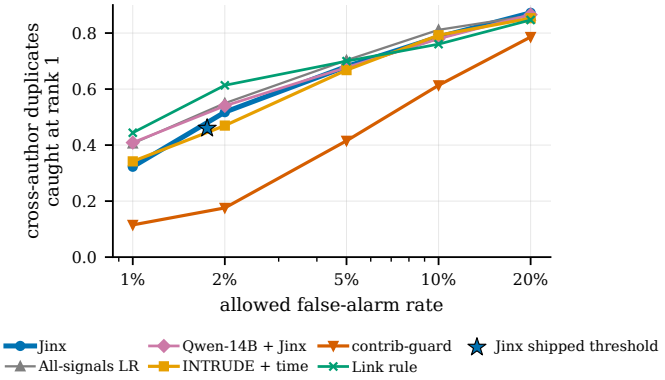


Fig. 5. Share of cross-author duplicates ranked first and flagged, for a fixed false-alarm budget on PRs without a duplicate. Thresholds are set on training folds; the curves are descriptive (no CIs). Star: Jinx at its shipped threshold.

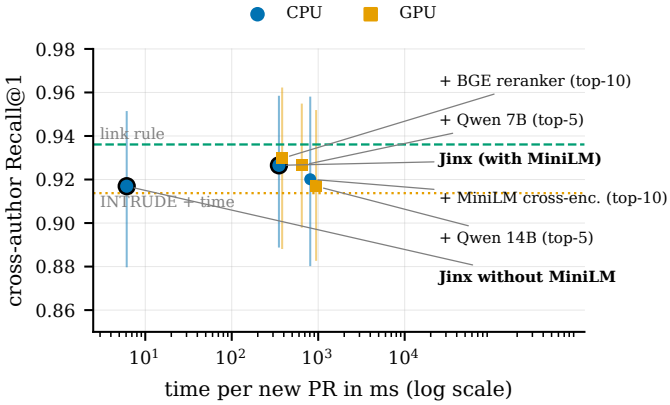


Fig. 6. Cross-author Recall@1 (with 95% CI) against time per new PR (median, log scale). Jinx times are for the installed package on 4 CPU threads, excluding GitHub API calls and model loading; added models are measured on top of Jinx with MiniLM. Differences in Recall@1 are within noise.

valid the comparison with INTRUDE + time does not change (Recall@1 difference +0.8 points, CI -1.3 to +2.7).

Links. People who close a duplicate often link the other PR, which makes link signals look strong. We count only links in the new PR's own text and report pairs with no link.

Queue design. Our first benchmark used the 7-day queue, which shows how easily such a choice slips in. Other choices may matter too; the 200-PR cap alone hides 11 cross-author duplicates.

Generality. We use one dataset and only agent PRs, so we cannot compare agent and human duplicates. Two bots were re-implemented and may behave differently from the deployed versions. The cost and replay analyses use historical comments only; no detector was run on live repositories.

VIII. Related Work

Li et al. detected duplicate PRs with TF-IDF and ranked earlier PRs for each new PR [6], and released DupPR [14]. INTRUDE added code and issue signals [7], and later work added time [8], graph embeddings [9], LLM summaries [10] and issue-to-PR links [11]; see the review in [15] and studies

of competing PRs [16], [17]. These works studied PRs by people and did not test how queue construction affects results. Studies of failed agent PRs list duplicates among the main causes [2], [3], [4], [5]; others study conflicts between concurrent agent PRs [18], coordination before a PR exists [19] and cross-references between PRs [20]. For duplicate bug reports, realistic evaluation shrank reported gains [12], and CUPID combines an LLM with retrieval [21].

IX. Conclusion

Duplicate agent PRs cost maintainers real attention, and a warning when the PR opens could save much of it. But which detector looks best depends on how the evaluation is built. Random pairs make strong methods look alike, and keeping recently closed PRs in the queue rewarded a signal that only helps when closed PRs are in the queue. On the open-only queue, a learned combination, a classic detector, plain TF-IDF and a simple link rule rank duplicates about equally well; they differ mainly in false alarms. We recommend that duplicate-PR studies build queues exactly as the tool will see them, report false alarms with confidence intervals, and report a simple link rule as a baseline.

Data Availability

The benchmark, code, raw scores and logs will be released under open licences with the camera-ready version; the Jinx tool is described separately [13].

References

[1] H. Li *et al.*, "AIDev: Studying AI coding agents on GitHub," 2026.

[2] R. Ehsani *et al.*, "Where do AI coding agents fail? an empirical study of failed agentic pull requests in GitHub," 2026, mSR 2026.

[3] K. Alam, S. Mondal, and B. Roy, "Why are AI agent involved pull requests (fix-related) remain unmerged?" arXiv:2602.00164, 2026.

[4] Nakashima *et al.*, "Why agentic-PRs get rejected: A comparative study of coding agents," 2026.

[5] M. Abujadallah, A. Arabat, and M. Sayagh, "Understanding the rejection of fixes generated by agentic pull requests: Insights from the AIDev dataset," arXiv:2606.13468, 2026.

[6] Z. Li, G. Yin, Y. Yu, T. Wang, and H. Wang, "Detecting duplicate pull-requests in GitHub," in *Proc. Internetware*, 2017.

[7] L. Ren, S. Zhou, C. Kästner, and A. Wąsowski, "Identifying redundancies in fork-based development," in *Proc. SANER*, 2019.

[8] Q. Wang, B. Xu, X. Xia, T. Wang, and S. Li, "Duplicate pull request detection: When time matters," in *Proc. Internetware*, 2019.

[9] J. Jiang, Q. Li, J. Guo, Z. Li, L. Zhang, and G. Li, "DupHunter: Detecting duplicate pull requests in fork-based development," *IEEE Trans. Softw. Eng.*, 2023.

[10] Z. Liao, Y. Liu, Y. Lan, and Y. Sun, "DupLLM: Duplicate pull requests detection based on large language model," in *Proc. APSEC*, 2024.

[11] R. Galbraith and D. Greer, "Detecting duplicate effort in GitHub contributions," in *Proc. ENASE*, 2025.

[12] T. Zhang, D. Han, V. Vinayakarao, I. C. Irsan, B. Xu, F. Thung, D. Lo, and L. Jiang, "Duplicate bug report detection: How far are we?" *ACM Trans. Softw. Eng. Methodol.*, 2023.

[13] H. Jain, S. Yergattikar, and T. Sah, "Jinx: Explainable duplicate warnings for pull requests from coding agents," 2026, tool demonstration paper, under review.

[14] Y. Yu, Z. Li, G. Yin, T. Wang, and H. Wang, "A dataset of duplicate pull-requests in GitHub," in *Proc. MSR*, 2018.

[15] B. Chekaya, Garrouch, and Omri, "Duplicate pull requests in code management platforms: A systematic literature review," Research Square preprint, 2026.

[16] X. Zhang, Y. Chen, Y. Gu, W. Zou, X. Xie, and X. Jia, "How do multiple pull requests change the same code: A study of competing pull requests in GitHub," in *Proc. ICSME*, 2018.

[17] Z. Li, Y. Yu, M. Zhou, T. Wang, and G. Yin, "Redundancy, context and preference: An empirical study of duplicate pull requests in OSS projects," *IEEE Trans. Softw. Eng.*, 2022.

[18] G. Xu, A. Subramanian, and N. Karthik, "AI agent pull requests on GitHub: Frequency, structure and merge conflict rates," arXiv:2607.04697, 2026.

[19] D. Sarkar, "Before the pull request: Mining multi-agent coordination," arXiv:2606.19616, 2026.

[20] Khemissi *et al.*, "Humans integrate, agents fix: How agent-authored pull requests are referenced in practice," 2026.

[21] T. Zhang, I. C. Irsan, F. Thung, and D. Lo, "CUPID: Leveraging ChatGPT for more accurate duplicate bug report detection," arXiv:2308.10022, 2023.